



HANDS-ON PROJECT • BUILD & LEARN • RDBMS

Airline Reservation System

Database Project

Learn every core database concept by building a real airline booking system

Step-by-Step • Full SQL • Sample Data • Exercises • Queries

Relationships

vs

ACID

Columns

Primary Keys

Foreign Keys

Schema

Indexes

Constraints

Airports • Airlines • Flights • Passengers • Bookings • Seats • Crew • Baggage • Payments

Section 1 — Core Concepts

What You Will Build

You will build a complete **Airline Reservation System** database — the kind powering real booking platforms like IndiGo, Air India, or Emirates. You will manage airports, airlines, flights, passengers, bookings, seats, crew, baggage, and payments. Every step introduces one or more core database concepts through hands-on SQL.

Phase	Step	What You Build	Concept Learned
1 — Design	Step 1	Identify 11 entities and map relationships	Tables, Schema
2 — Create	Step 2	Create database and parent tables	Columns, Data Types
2 — Create	Step 3	Primary Keys on all tables	Primary Key
2 — Create	Step 4	Foreign Keys — link all tables	Foreign Key, Relationships
2 — Create	Step 5	Constraints for business rules	Constraints (all 6 types)
3 — Populate	Step 6	Insert airports, airlines, flights, passengers	Rows / INSERT
3 — Populate	Step 7	Insert bookings, seats, crew, baggage, payments	Multi-table INSERT
3 — Populate	Step 8	Real UPDATE and DELETE scenarios	UPDATE, DELETE
4 — Optimise	Step 9	Create indexes for fast flight searches	Index, EXPLAIN
5 — Query	Step 10	Queries — passenger history, seat map, revenue	SELECT, JOINS
6 — Transact	Step 11	Transactions for booking and payment	ACID Properties
7 — Practice	Exercises	20 tasks + checklist	All Concepts

PHASE 1

Schema Design

Identify every real-world entity in an airline system and plan their connections

Concepts covered: Tables • Schema • Relationships (1:1, 1:N, M:N)

Step 1 — Identify Entities and Relationships

Every **noun** that the system needs to track becomes a **table**. Start by listing all the things an airline booking system must store.

Entity (Table)	What It Stores	Example Data
Airports	All airports in the network	Chennai (MAA), Delhi (DEL), Dubai (DXB)
Airlines	Airline companies	IndiGo (6E), Air India (AI), Emirates (EK)
Aircraft	Planes owned by airlines	Boeing 737, Airbus A320 — seats, range
Flights	Scheduled flight instances	6E-201 MAA→DEL, departs 06:00, 2024-07-01
FlightSeats	Every seat on every flight	Row 1A, 12C — class, status (free/booked)
Passengers	Registered travellers	Name, passport, nationality, DOB
Bookings	A reservation by a passenger for a flight	PNR, booking date, status
BookingSeats	Seat assigned per booking (M:N bridge)	BookingID + SeatID + meal preference
CrewMembers	All airline staff (pilots, cabin crew)	Name, role, licence, airline
FlightCrew	Crew assigned to a flight (M:N bridge)	FlightID + CrewID + role on that flight
Baggage	Checked bags per booking	Tag, weight, status — checked in / loaded
Payments	Payment records per booking	Amount, mode, transaction ID, status

Relationship Map

Table A	Relationship	Table B	Type	Implementation
Airlines	operates many	Aircraft	1:N	AirlineID FK in Aircraft
Airlines	operates many	Flights	1:N	AirlineID FK in Flights
Airports	is origin of	Flights	1:N	OriginAirportID FK in Flights
Airports	is dest of	Flights	1:N	DestAirportID FK in Flights
Aircraft	used in many	Flights	1:N	AircraftID FK in Flights
Flights	has many	FlightSeats	1:N	FlightID FK in FlightSeats
Passengers	makes many	Bookings	1:N	PassengerID FK in Bookings
Bookings	covers many	FlightSeats	M:N	BookingSeats junction table

Table A	Relationship	Table B	Type	Implementation
Airlines	employs many	CrewMembers	1:N	AirlineID FK in CrewMembers
Flights	staffed by	CrewMembers	M:N	FlightCrew junction table
Bookings	has many	Baggage	1:N	BookingID FK in Baggage
Bookings	has one	Payments	1:1	BookingID UNIQUE FK in Payments

Schema Diagram

SCHEMA DIAGRAM

```
[Airlines] ■■1:N■■ [Aircraft] ■■1:N■■ [Flights] ■■1:N■■ [FlightSeats]
■ ■ ■
1:N M:N M:N (via [BookingSeats])
■ (via [FlightCrew]) ■
[CrewMembers] [CrewMembers] [Bookings]
■
[Airports] ■■1:N■■ [Flights] [Passengers] ■■1:N■■ [Bookings]
(Origin + Dest) ■ ■
[Baggage] [Payments] (1:1)
```

PHASE 2

Concepts covered: Columns • Data Types • Primary Key • Foreign Key • Constraints

[illegible]

Step 4 — Bookings, Junction Tables, Baggage, Payments

[illegible]

PHASE 3

Concepts covered: Rows • INSERT INTO • Multi-table Population

[illegible]

Step 7 — Bookings, Crew, Baggage, Payments

Step 8 — UPDATE and DELETE Scenarios

Page 7

```
FlightHours = FlightHours + 2 WHERE CrewID IN (1, 2, 3); -- after 6E-201 completes (MAA→DEL = 2.5h) -- DELETE: Passenger cancels booking (cascade removes payment + baggage) DELETE FROM Bookings WHERE PNR = 'GH3456' AND Status = 'Confirmed'; -- Cascade: BookingSeats, Baggage, Payment for this booking also deleted -- Soft delete: cancel a flight (don't delete – keep booking history) UPDATE Flights SET Status = 'Cancelled' WHERE FlightID = 8; -- Then notify all passengers via application logic
```

Add Indexes

PHASE 4

Speed up flight searches, seat lookups and passenger queries

Concepts covered: Index • EXPLAIN • Query Performance

Step 9 — Create Indexes for Common Airline Queries

```
-- ■■ Index 1: Search flights by route (origin + destination) ■■■■■■■■■■ CREATE INDEX
idx_flight_route ON Flights(OriginID, DestinationID); -- ■■ Index 2: Search flights by date
(most common booking query) ■■■■■■ CREATE INDEX idx_flight_date ON Flights(DepartureTime);
-- ■■ Index 3: Composite – route + date (covers the main search) ■■■■■■ CREATE INDEX
idx_flight_route_date ON Flights(OriginID, DestinationID, DepartureTime); -- ■■ Index 4:
Find booking by PNR (every check-in lookup) ■■■■■■■■■■ CREATE UNIQUE INDEX
idx_booking_pnr ON Bookings(PNR); -- ■■ Index 5: Passenger by passport (immigration /
boarding) ■■■■■■■■■■ CREATE UNIQUE INDEX idx_pax_passport ON Passengers(Passport); -- ■■
Index 6: Available seats on a flight ■■■■■■■■■■ CREATE INDEX
idx_seat_avail ON FlightSeats(FlightID, IsAvailable, Class); -- ■■ Index 7: Baggage by tag
(ground handling scan) ■■■■■■■■■■ CREATE UNIQUE INDEX idx_bag_tag ON
Baggage(TagNo); -- ■■ Index 8: Payments by status (finance dashboard)
■■■■■■■■■■■ CREATE INDEX idx_pay_status ON Payments(Status, PaidOn); -- ■■
Verify index is used ■■■■■■■■■■ EXPLAIN
SELECT FlightNo, DepartureTime, BasePrice FROM Flights WHERE OriginID = 1 AND DestinationID
= 2 AND DepartureTime >= '2024-07-15' AND Status = 'Scheduled'; -- key:
idx_flight_route_date type: range ← index used!
```

Index	Columns	Query Served	Benefit
idx_flight_route_date	OriginID+DestID+DeptTime	Availability search: MAA→DEL on Jul 15	Core booking query, O(log n)
idx_booking_pnr	PNR (UNIQUE)	Check-in, cancellation lookup	Instant PNR lookup
idx_pax_passport	Passport (UNIQUE)	Immigration, boarding gate scan	O(1) passport match
idx_seat_avail	FlightID+IsAvailable+Class	Find free Business seats on flight	Avoids full seat scan
idx_bag_tag	TagNo (UNIQUE)	Baggage belt scanner lookup	Real-time tag scan
idx_pay_status	Status+PaidOn	Finance: all Pending payments today	Filtered date-range scan

Query the Database

PHASE 5

Real airline queries — search, seat map, revenue, passenger history

Concepts covered: **SELECT** • **JOIN** • **GROUP BY** • **Subqueries**

Step 10 — Essential Airline Queries

```
-- Q1: Flight availability search – Chennai to Delhi on 15 July
SELECT f.FlightNo, al.Name AS Airline,
CONCAT(ap1.IATA, ' → ', ap2.IATA) AS Route, f.DepartureTime, f.ArrivalTime,
TIMESTAMPDIFF(MINUTE, f.DepartureTime, f.ArrivalTime) AS DurationMins, f.BasePrice,
f.Status, COUNT(CASE WHEN fs.IsAvailable=TRUE THEN 1 END) AS SeatsLeft
FROM Flights f JOIN Airlines al ON al.AirlineID = f.AirlineID
JOIN Airports ap1 ON ap1.AirportID = f.OriginID
JOIN Airports ap2 ON ap2.AirportID = f.DestinationID
LEFT JOIN FlightSeats fs ON fs.FlightID = f.FlightID
WHERE f.OriginID = 1 AND f.DestinationID = 2 AND DATE(f.DepartureTime) =
'2024-07-15' AND f.Status = 'Scheduled'
GROUP BY f.FlightID ORDER BY f.DepartureTime;

-- Q2: Seat map for a flight – show all seats with availability
SELECT fs.SeatNo, fs.Class, fs.Price,
CASE WHEN fs.IsAvailable THEN 'FREE' ELSE 'BOOKED' END AS SeatStatus,
p.FullName AS PassengerName
FROM FlightSeats fs LEFT JOIN BookingSeats bs ON bs.SeatID = fs.SeatID
LEFT JOIN Bookings b ON b.BookingID = bs.BookingID
LEFT JOIN Passengers p ON p.PassengerID = b.PassengerID
WHERE fs.FlightID = 1 ORDER BY fs.Class, fs.SeatNo;

-- Q3: Full passenger travel history
SELECT p.FullName, p.Passport, f.FlightNo, al.Name AS Airline,
CONCAT(ap1.City, ' → ', ap2.City) AS Route, f.DepartureTime,
b.TotalFare, b.Status AS BookingStatus
FROM Passengers p JOIN Bookings b ON b.PassengerID = p.PassengerID
JOIN Flights f ON f.FlightID = b.FlightID
JOIN Airlines al ON al.AirlineID = f.AirlineID
JOIN Airports ap1 ON ap1.AirportID = f.OriginID
JOIN Airports ap2 ON ap2.AirportID = f.DestinationID
WHERE p.PassengerID = 1 ORDER BY f.DepartureTime;
```

Aggregate Queries — Revenue and Operations

```
-- Q4: Revenue per airline – total collected
SELECT al.Name AS Airline, COUNT(b.BookingID) AS TotalBookings,
SUM(py.Amount) AS TotalRevenue, AVG(py.Amount) AS AvgTicketPrice
FROM Airlines al JOIN Flights f ON f.AirlineID = al.AirlineID
JOIN Bookings b ON b.FlightID = f.FlightID
JOIN Payments py ON py.BookingID = b.BookingID
WHERE py.Status = 'Success'
GROUP BY al.AirlineID, al.Name ORDER BY TotalRevenue DESC;

-- Q5: Most popular routes (by number of bookings)
SELECT CONCAT(ap1.City, ' → ', ap2.City) AS Route,
COUNT(b.BookingID) AS TotalBookings, SUM(b.TotalFare) AS RouteRevenue
FROM Bookings b JOIN Flights f ON f.FlightID = b.FlightID
JOIN Airports ap1 ON ap1.AirportID = f.OriginID
JOIN Airports ap2 ON ap2.AirportID = f.DestinationID
GROUP BY f.OriginID, f.DestinationID ORDER BY TotalBookings DESC
LIMIT 10;

-- Q6: Crew flying hours – top 5 most active crew
SELECT cm.FullName, cm.Role, al.Name AS Airline, cm.FlightHours,
COUNT(fc.FlightID) AS FlightsAssigned
FROM CrewMembers cm JOIN Airlines al ON al.AirlineID = cm.AirlineID
LEFT JOIN FlightCrew fc ON fc.CrewID = cm.CrewID
GROUP BY cm.CrewID ORDER BY cm.FlightHours DESC
LIMIT 5;
```

Transactions & ACID

PHASE 6

Seat booking, payment, and cancellation must be atomic and isolated

Concepts covered: ACID • BEGIN • COMMIT • ROLLBACK • SAVEPOINT • FOR UPDATE

Transaction 1: Seat Booking — Atomicity + Isolation

Booking a seat involves checking availability, creating a booking, assigning the seat, and recording payment. **All steps must succeed or all must fail.**

```
START TRANSACTION; -- Step A: Lock the seat row to prevent double-booking (ISOLATION) SELECT
SeatID, IsAvailable, Price FROM FlightSeats WHERE FlightID = 1 AND SeatNo = '30A' AND
IsAvailable = TRUE FOR UPDATE; -- row lock: no other session can modify this seat until
COMMIT -- If 0 rows returned (seat taken) → ROLLBACK immediately -- Step B: Create the
booking (ATOMICITY) INSERT INTO Bookings (PNR, PassengerID, FlightID, TotalFare, Status)
VALUES ('QR5678', 6, 1, 2900.00, 'Confirmed'); SET @new_booking = LAST_INSERT_ID(); -- Step
C: Assign the seat INSERT INTO BookingSeats (BookingID, SeatID, MealPref) VALUES
(@new_booking, 13, 'Veg'); -- SeatID 13 = seat 30A -- Step D: Mark seat as booked UPDATE
FlightSeats SET IsAvailable = FALSE WHERE SeatID = 13; -- Step E: Record payment INSERT INTO
Payments (BookingID, Amount, Mode, TxnRef, Status) VALUES (@new_booking, 2900.00, 'UPI',
'UPI20240701099', 'Success'); COMMIT; -- All 5 steps permanent (DURABILITY) -- If payment
gateway fails at Step E: -- ROLLBACK → seat reverts to available, booking deleted, no
orphan data
```

Transaction 2: Flight Cancellation with SAVEPOINT

When a flight is cancelled, the system must: cancel all bookings, release all seats, initiate refunds. **SAVEPOINT** lets us handle failures in stages.

```
START TRANSACTION; -- Step A: Cancel the flight UPDATE Flights SET Status = 'Cancelled'
WHERE FlightID = 6; SAVEPOINT flight_cancelled; -- Step B: Cancel all confirmed bookings for
this flight UPDATE Bookings SET Status = 'Cancelled' WHERE FlightID = 6 AND Status IN
('Confirmed','Checked-In'); SAVEPOINT bookings_cancelled; -- Step C: Release all seats back
to available UPDATE FlightSeats SET IsAvailable = TRUE WHERE FlightID = 6; SAVEPOINT
seats_released; -- Step D: Initiate refunds for paid bookings UPDATE Payments SET Status =
'Refunded' WHERE BookingID IN ( SELECT BookingID FROM Bookings WHERE FlightID = 6 ) AND
Status = 'Success'; -- If refund system is down, rollback only the refund step: -- ROLLBACK
TO SAVEPOINT seats_released; -- (Queue refunds for retry – seats and bookings still
cancelled) COMMIT;
```

Transaction 3: Passenger Upgrade — Consistency

```
START TRANSACTION; -- Find a free Business class seat SELECT SeatID FROM FlightSeats WHERE
FlightID=1 AND Class='Business' AND IsAvailable=TRUE LIMIT 1 FOR UPDATE; -- Remove old
Economy seat assignment UPDATE FlightSeats SET IsAvailable=TRUE WHERE SeatID = ( SELECT
bs.SeatID FROM BookingSeats bs JOIN FlightSeats fs ON fs.SeatID = bs.SeatID WHERE
bs.BookingID = 1 AND fs.Class = 'Economy' ); DELETE FROM BookingSeats WHERE BookingID=1 AND
SeatID IN (SELECT SeatID FROM FlightSeats WHERE FlightID=1 AND Class='Economy'); -- Assign
new Business seat INSERT INTO BookingSeats (BookingID, SeatID, MealPref) VALUES (1, 1,
'Veg'); -- SeatID 1 = Seat 1A Business UPDATE FlightSeats SET IsAvailable=FALSE WHERE
```

```
SeatID=1; -- Update fare difference UPDATE Bookings SET TotalFare = 8500.00 WHERE  
BookingID=1; COMMIT; -- Upgrade complete – consistency maintained (no double-booking)
```

Practice Exercises

PHASE 7

20 tasks covering every concept — try them on your Airline database!

Concepts covered: All Concepts • Self-Assessment • Real Scenarios

Table & Schema Tasks

1 Extend the schema

- Add a 'Lounges' table linked to Airports — LoungeID, Name, Class, Capacity
- Add a 'LoungeAccess' table (BookingID → Lounge) — what relationship type is this?
- Add a 'Meals' table and link it to FlightSeats (pre-ordered meal per seat)

2 Modify existing tables

- Add a 'FreqFlyerTier' ENUM('Blue','Silver','Gold','Platinum') column to Passengers
- Add a 'GateNo' VARCHAR(5) column to Flights for boarding gate info
- Widen FlightSeats.SeatNo from VARCHAR(4) to VARCHAR(5) for double-digit rows

Row Tasks

3 INSERT tasks

- Add an international flight: BOM → SIN operated by Singapore Airlines
- Register 3 new passengers and book them on flight 6E-301 (FlightID=6)
- Assign crew to flight 7 (AI-202) — a captain, co-pilot, and 2 cabin crew

4 UPDATE & DELETE tasks

- Update flight 6E-202 status to 'Boarding' and set its gate to 'B12'
- A passenger changed meal preference from Veg to Jain — write the UPDATE
- Cancel booking PNR 'MN6789' — does it cascade to Baggage and Payment? Verify

Constraint Tasks

5 Test constraints

- Insert a flight where ArrivalTime is before DepartureTime — which CHECK fails?
- Insert a flight with OriginID = DestinationID — which constraint blocks it?
- Try booking the same seat twice on the same flight — which UNIQUE constraint fires?
- Insert baggage with WeightKg = 55 — which CHECK constraint rejects it?

Index Tasks

6 Index work

- Run EXPLAIN on the Q1 flight search query — confirm idx_flight_route_date is used
- Run EXPLAIN on: SELECT * FROM Baggage WHERE TagNo='TAG001234' — index used?
- Should you index FlightSeats.Class alone? Think about cardinality and explain why not

Query Tasks

7 Write these SELECT queries

- Show all passengers on a specific flight with their seat number and meal preference
- Find all flights with more than 80% of seats still available
- List the top 5 passengers by total spend (sum of all their booking fares)

8 Advanced queries

- Find flights where the captain has fewer than 5000 total flight hours (safety check)
- Show connecting flight options: MAA→DXB→LHR with layover less than 6 hours
- Monthly revenue report: group by airline and month, show total and average fare

Transaction Tasks

9 ACID scenarios

- Write a booking transaction that rolls back if payment fails — verify with SELECT
- Simulate two passengers booking the last seat at the same time — show FOR UPDATE preventing double-booking
- Use SAVEPOINT in a group booking (4 passengers) — rollback if passenger 3 has invalid passport
- Write a full refund transaction: cancel booking, release seat, refund payment, log it

Self-Assessment Checklist

PRACTICE CHECKLIST

- ☐ Create all 12 tables in the correct FK dependency order
- ☐ Define columns with ENUM, DECIMAL, DATETIME, CHAR across multiple tables
- ☐ Create the composite UNIQUE constraint: (FlightNo, DepartureTime) in Flights
- ☐ Create a 1:N relationship (Airlines → Flights) with ON DELETE RESTRICT
- ☐ Create a M:N relationship (Flights ↔ CrewMembers via FlightCrew)
- ☐ Create a 1:1 relationship (Bookings → Payments) using UNIQUE FK
- ☐ Insert all airports, airlines, aircraft, and 8 flights
- ☐ Assign seats to bookings and verify IsAvailable updates correctly
- ☐ Use EXPLAIN to confirm idx_flight_route_date is used in flight search
- ☐ Write a 5-table JOIN query (Passengers + Bookings + Flights + Airlines + Airports)
- ☐ Write a GROUP BY with HAVING (airlines with revenue > 10000)
- ☐ Use a subquery to find passengers who never booked a Business class seat
- ☐ Write a booking transaction (5 INSERT/UPDATE steps) with BEGIN/COMMIT
- ☐ Trigger a ROLLBACK when the seat is already taken — verify no partial data
- ☐ Use SAVEPOINT in a flight cancellation to handle refund failures gracefully

Quick Reference — All 12 Tables

Table	Primary Key	Key FKs	Key Constraints	Relationship
Airports	AirportID AUTO	—	IATA UNIQUE, Terminals>0	Root parent
Airlines	AirlineID AUTO	—	IATA UNIQUE, Name UNIQUE	Root parent
Aircraft	AircraftID AUTO	AirlineID → Airlines	TailNo UNIQUE, Seats 50-900	N:1 Airlines
Flights	FlightID AUTO	AirlineID, AircraftID, OriginID, DestID	Arrival>Departure, Origin≠Dest	N:1 each
FlightSeats	SeatID AUTO	FlightID → Flights	UNIQUE(FlightID,SeatNo), Price>0	1:N Flights
Passengers	PassengerID AUTO	—	Passport UNIQUE, Email UNIQUE	One entity
Bookings	BookingID AUTO	PassengerID, FlightID	PNR UNIQUE, TotalFare>0	N:1 each
BookingSeats	BookingID+SeatID	BookingID, SeatID	MealPref ENUM, composite PK	M:N junction
CrewMembers	CrewID AUTO	AirlineID → Airlines	LicenceNo UNIQUE, Hours≥0	N:1 Airlines
FlightCrew	FlightID+CrewID	FlightID, CrewID	RoleOnFlight ENUM, composite PK	M:N junction
Baggage	BagID AUTO	BookingID → Bookings	TagNo UNIQUE, Weight 0-50	N:1 Bookings
Payments	PaymentID AUTO	BookingID → Bookings (UNIQUE)	Invoice UNIQUE, Amount>0	1:1 Bookings

Concept Map — What Each Phase Taught You

Concept	Phase/Step	What You Did
Table	Phase 1	Identified 12 airline entities; each became a table
Schema	Phase 1+2	Drew relationship map with 12 FK links; wrote full DDL script
Column	Phase 2	Used ENUM, DECIMAL, DATETIME, BOOLEAN, CHAR across all tables
Primary Key	Phase 2	AUTO_INCREMENT PK; composite PK on BookingSeats and FlightCrew
Foreign Key	Phase 2	12 FK relationships; RESTRICT, SET NULL, CASCADE rules defined
Constraints	Phase 2+3	CHECK (arrival>departure, origin≠dest), UNIQUE (PNR, TailNo, TagNo)
Rows	Phase 3	8 airports, 6 airlines, 7 aircraft, 8 flights, 8 passengers, bookings
Index	Phase 4	8 indexes; composite route+date index verified with EXPLAIN
Relationships	Phase 2	1:N (Airlines→Flights), 1:1 (Bookings→Payments), M:N (Flights↔Crew)
ACID	Phase 6	3 transactions: booking (Atomicity+Isolation), cancellation (SAVEPOINT), upgrade

Extend the Project

Extension	New Tables	New Concepts
Loyalty Programme	FreqFlyerAccounts, MileTransactions	Triggers for auto-awarding miles on booking

Extension	New Tables	New Concepts
Dynamic Pricing	PriceRules, DemandLog	CHECK constraints; time-based price logic
Maintenance Tracking	MaintenanceLogs, AircraftParts	Scheduled tasks; M:N aircraft-parts
Check-in Kiosk	CheckInSessions, BoardingPasses	Generated columns; UUID for pass codes
Analytics Dashboard	(Views + CTEs)	CREATE VIEW for seat occupancy, revenue
Cargo Booking	CargoShipments, CargoItems	Separate 1:N chain; weight constraints